

EDA

This notebook cleans the Chocolate Sales dataset and creates 2 static visuals to support our dashboard user story (sales trends over time by country).

Why we do these checks

Before making plots, we do quick data checks that keep the dashboard stable: we confirm data types, check missing values and duplicates, and make sure key fields like date and sales are usable. We also create simple Year/Month fields because most dashboard views (monthly trends and filtering) depend on them.

```
In [1]: import pandas as pd
import altair as alt

DATA_PATH = "../data/raw/chocolate-sales.csv"

df_raw = pd.read_csv(DATA_PATH)

# Make column names lowercase + underscores
df_raw.columns = (
    df_raw.columns.str.strip().str.lower().str.replace(" ", "_")
)

# rename amount with sales
df_raw = df_raw.rename(columns={"amount": "sales"})
df_raw.head()
```

```
Out[1]:
```

	sales_person	country	product	date	sales	boxes_shipped
0	Jehu Rudeforth	UK	Mint Chip Choco	04/01/2022	\$5,320.00	180
1	Van Tuxwell	India	85% Dark Bars	01/08/2022	\$7,896.00	94
2	Gigi Bohling	India	Peanut Butter Cubes	07/07/2022	\$4,501.00	91
3	Jan Morforth	Australia	Peanut Butter Cubes	27/04/2022	\$12,726.00	342
4	Jehu Rudeforth	UK	Peanut Butter Cubes	24/02/2022	\$13,685.00	184

```
In [2]: display(df_raw.shape)
```

```
(3282, 6)
```

```
In [3]: df_raw.info()
display(df_raw.isna().sum())
```

```
display(df_raw.duplicated().sum())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3282 entries, 0 to 3281
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sales_person    3282 non-null   str
1   country         3282 non-null   str
2   product         3282 non-null   str
3   date            3282 non-null   str
4   sales           3282 non-null   str
5   boxes_shipped   3282 non-null   int64
dtypes: int64(1), str(5)
memory usage: 154.0 KB
sales_person    0
country         0
product         0
date            0
sales           0
boxes_shipped   0
dtype: int64
np.int64(0)
```

```
In [4]: # how many unique values per column
display(df_raw.nunique())

# Country categories
display(sorted(df_raw["country"].unique()))
```

```
sales_person    25
country         6
product         22
date            504
sales           3013
boxes_shipped   507
dtype: int64
['Australia', 'Canada', 'India', 'New Zealand', 'UK', 'USA']
```

```
In [5]: # Parse date so we can do time trends
df = df_raw.copy()
df["date"] = pd.to_datetime(df["date"], format="%d/%m/%Y", errors="coerce")
df["date"].isna().sum()
```

```
Out[5]: np.int64(0)
```

```
In [6]: # Convert sales to numeric for sums/plots
df["sales"] = (
    df["sales"].astype(str)
    .str.replace("$", "", regex=False)
    .str.replace(",", "", regex=False)
)
df["sales"] = pd.to_numeric(df["sales"], errors="coerce")
df["sales"].isna().sum()
```

```
Out[6]: np.int64(0)
```

```
In [7]: # Time features
df["year"] = df["date"].dt.year
df["year_month_period"] = df["date"].dt.to_period("M")
df["year_month"] = df["year_month_period"].astype(str)

# Month name
df["month_name"] = df["date"].dt.strftime("%B")
```

```
In [8]: # Check which months appear to confirm coverage is Jan-Aug
df["month_num"] = df["date"].dt.month

months_present = (
    df[["month_num", "month_name"]]
    .dropna()
    .drop_duplicates()
    .sort_values("month_num")
)

display(months_present["month_name"].tolist())
```

```
['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August']
```

```
In [9]: # Save cleaned dataset
from pathlib import Path

out_dir = Path("../") / "data" / "processed"
out_dir.mkdir(parents=True, exist_ok=True)

out_path = out_dir / "chocolate_sales_clean.csv"
#df.to_csv(out_path, index=False)

out_path
```

```
Out[9]: PosixPath('../data/processed/chocolate_sales_clean.csv')
```

EDA for User Story: Sales trends over time by country

User Story: As a sales manager, I want to view sales trends over time by country so I can identify which markets are growing or declining.

Visual 1: Quarterly sales trend by country

```
In [10]: # Aggregate to quarterly totals (less noisy than monthly, easier to compare
df["quarter_start"] = df["date"].dt.to_period("Q").dt.start_time
df["quarter_label"] = df["date"].dt.to_period("Q").astype(str) # e.g., 2022

quarterly = (
    df.groupby(["quarter_start", "quarter_label", "country"], as_index=False)
    .agg(total_sales=("sales", "sum"))
    .sort_values(["quarter_start", "country"])
)
```

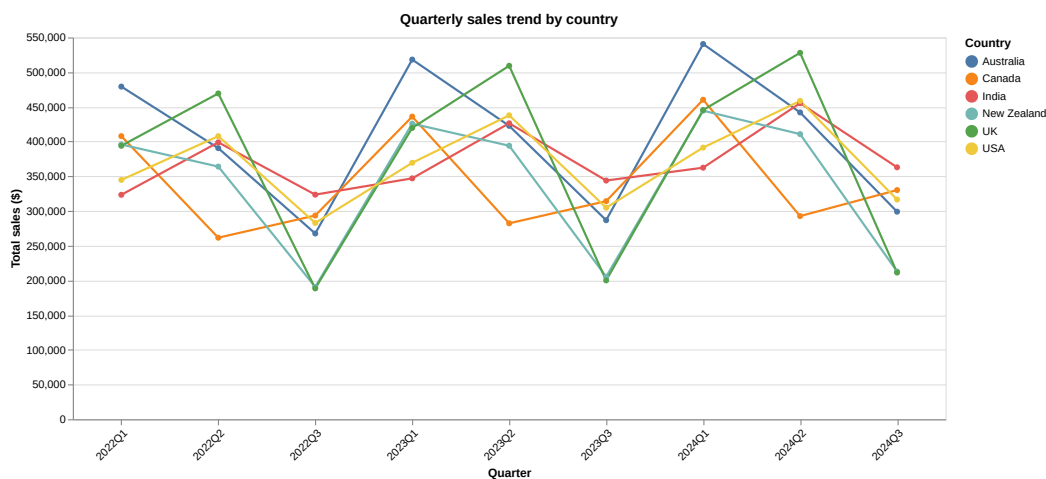
```

chart_q = (
    alt.Chart(quarterly)
    .mark_line(point=True)
    .encode(
        x=alt.X(
            "quarter_label:N",
            sort=alt.SortField(field="quarter_start", order="ascending"),
            axis=alt.Axis(title="Quarter", labelAngle=-45),
        ),
        y=alt.Y("total_sales:Q", axis=alt.Axis(title="Total sales ($)"), form
        color=alt.Color("country:N", legend=alt.Legend(title="Country")),
        tooltip=[
            alt.Tooltip("country:N", title="Country"),
            alt.Tooltip("quarter_label:N", title="Quarter"),
            alt.Tooltip("total_sales:Q", title="Total sales", format=".0f")
        ],
    )
    .properties(title="Quarterly sales trend by country", width=800, height=
)

chart_q

```

Out[10]:



We aggregated sales to the quarterly level to reduce month-to-month noise and make trends easier to compare across countries. This chart helps a sales manager quickly see which countries are generally rising or falling over time, which supports decisions about which markets to focus on.

Visual 2: Year-over-year sales growth by country (Jan–Aug 2024 vs Jan–Aug 2023)

```

In [11]: # Compare Jan–Aug 2024 vs Jan–Aug 2023 (dataset covers Jan–Aug, so we use th
df["year"] = df["date"].dt.year
df["month_num"] = df["date"].dt.month

yoy_totals = (
    df[df["year"].isin([2023, 2024]) & df["month_num"].between(1, 8)]
    .groupby(["country", "year"], as_index=False)
    .agg(total_sales=("sales", "sum"))
)

```

```

)

wide = yoy_totals.pivot(index="country", columns="year", values="total_sales")

# Rename pivot year columns to clean string names
wide = wide.rename(columns={2023: "sales_2023", 2024: "sales_2024"})

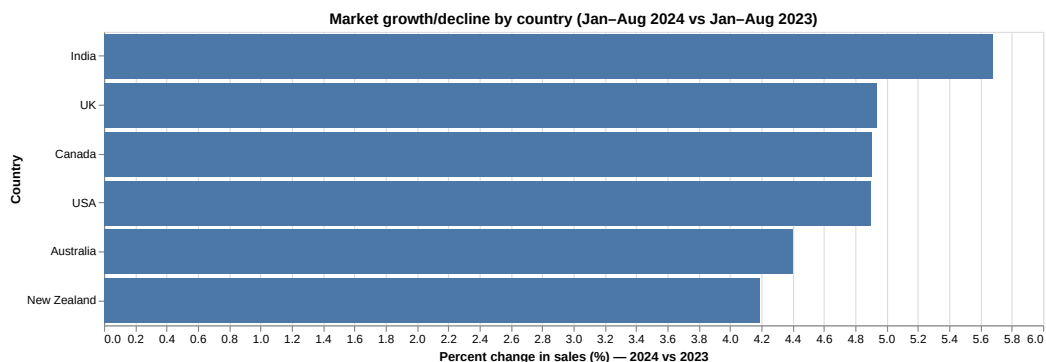
wide["pct_change"] = (wide["sales_2024"] - wide["sales_2023"]) / wide["sales_2023"]

growth_chart = (
    alt.Chart(wide)
    .mark_bar()
    .encode(
        y=alt.Y("country:N", sort="-x", title="Country"),
        x=alt.X("pct_change:Q", title="Percent change in sales (%) – 2024 vs 2023",
            tooltip=[
                alt.Tooltip("country:N", title="Country"),
                alt.Tooltip("sales_2023:Q", title="2023 total", format=",.0f"),
                alt.Tooltip("sales_2024:Q", title="2024 total", format=",.0f"),
                alt.Tooltip("pct_change:Q", title="% change", format=",.2f"),
            ],
        ),
    ).properties(title="Market growth/decline by country (Jan–Aug 2024 vs Jan–Aug 2023)")

growth_chart

```

Out[11]:



This bar chart ranks countries by the year-over-year percent change in total sales (Jan–Aug 2024 vs Jan–Aug 2023). It helps the sales manager quickly identify which markets are growing fastest (e.g., India) and which are growing more slowly (e.g., New Zealand), which supports prioritizing where to focus sales efforts.